Sansiro Perfume App — Project Plan

Team: Alvin, Brandon, Chris

Timeline: 4 weeks, start to launch

Goal: Clients see what perfumes are in stock, order through the site, order lands as a ready-to-send WhatsApp message. No logins, no payments, no forms to fill in.

Stack: Next.js (React) + Tailwind, Neon (Postgres), hosted on Cloudflare Workers, all data access going through a backend API layer — no direct DB calls from the browser.

Live: https://sansiro-webapp.amwaniki-am.workers.dev/ → moving to `sansiro.blacktech.co.ke` (Alvin)

Architecture

Browser (React/Tailwind)

fetch()

Next.js API Routes / Route Handlers ← backend layer

SQL via @neondatabase/serverless

Neon (Postgres)

The frontend never talks to Neon directly. It calls our own endpoints (`GET /api/products`, `POST /api/orders`), and those route handlers hold the DB connection string and query logic.

Deployment mode: the `*.workers.dev` URL confirms the site's already on **Cloudflare Workers**. We stay there and use an adapter (OpenNext for Cloudflare, or equivalent) to run Next.js API routes as Workers — a build config change, not a platform migration.

Catalog & categorization — decided

~70 products, seeded from the catalog data already compiled (item code, designer, availability, scent group, notes). Two natural filters fall out of that data for free, no extra schema needed:

- **Gender** — derived from item code prefix (`K` = Ladies, `E` = Men, `M` = Men, `U` = Unisex)
- **Scent group** — already tagged per product (Floral, Oriental, Woody, Fresh-Chypre)

Catalog page can filter/group by either.

Database (4 tables, on Neon)

- `categories` — scent group (Floral / Oriental / Woody / Fresh-Chypre)
- `products` — name, designer, gender, description/notes, image
- `product_variants` — size, price, availability (per size)
- `orders` + `order_items` — a log of what was sent to WhatsApp (no customer name/phone/location — that lives in the WhatsApp conversation, not the DB)

No customer accounts. No stock deduction trigger on order creation — `product_variants.quantity` only changes via manual edit in Neon's SQL console.

Ordering

"Order via WhatsApp" button opens `wa.me/254719712242?text=<cart summary>` — all orders route to Alvin's WhatsApp number (**+254 719 712242**) for now.

Who's doing what

Alvin

- Neon project setup + schema
- Cloudflare Workers deployment config + custom domain (`sansiro.blacktech.co.ke`)
- Manual stock updates, ongoing (day-to-day owner)

Brandon

- Catalog page UI (product cards, stock badges, gender/scent filters)
- Cart UI
- Mobile responsiveness

Chris

- Next.js API route handlers (`/api/products`, `/api/orders`), querying Neon
- Stock status logic (server-side)
- Cart logic (client-side state, feeding into the order flow)
- WhatsApp message generator + `wa.me` link

Timeline (4 weeks)

Week 1 — Foundation

Day	Task	Owner
1–2	Neon project + schema (categories, products, variants, orders, order_items)	Alvin
1–2	Switch build from static export → Workers adapter	Alvin
3–4	Seed ~70 products from catalog data	Alvin
3–5	Point `sansiro.blacktech.co.ke` at the Worker	Alvin
3–5	`/api/products` route (list + filter by gender/scent group)	Chris

End of week 1 checkpoint: DB live with real data, API returning products, custom domain resolving.

Week 2 — Catalog

Day	Task	Owner
1–3	Catalog page consuming `/api/products` — cards, stock badges	Brandon
1–3	Gender / scent group filters on catalog page	Brandon
2–4	`/api/products/:id` route for detail view	Chris
3–5	Product detail view/modal	Brandon
4–5	Mobile responsiveness pass on catalog	Brandon

End of week 2 checkpoint: Full catalog browsable on desktop + mobile, showing real stock status.

Week 3 — Cart & Ordering

Day	Task	Owner
1–2	Cart state logic (add/remove/update qty, block out-of-stock)	Chris
1–3	Cart UI (drawer/page)	Brandon
2–3	`/api/orders` route — logs order, no stock change	Chris
3–4	WhatsApp message generator from cart contents	Chris
4–5	"Order via WhatsApp" button → `wa.me` link wired end-to-end	Chris

End of week 3 checkpoint: Full flow works — browse → cart → WhatsApp message pre-filled and sent.

Week 4 — Test, Polish, Launch

Day	Task	Owner
1–2	Cross-device testing (iOS Safari, Android Chrome) for `wa.me` deep links	All
1–2	Empty states (no results after filter, all out of stock, etc.)	Brandon
3	Bug fixes from testing	All
4	Final deploy to `sansiro.blacktech.co.ke`	Alvin
5	Handoff walkthrough — how Alvin updates stock in Neon	Alvin

Launch: end of week 4.

Not doing yet

- Auto stock deduction on order (manual only — Alvin updates it)
- Payments
- Customer accounts / order history for customers
- WhatsApp Business API / automated replies
- Auth on API routes (fine for v1 since routes are read-mostly and low-risk)